Monday

May Day Bank Holiday (UK, Republic of Ireland)

1 - 20
2 - 21
~~31~~ - 22 May' 16
8 - 23
16 - 24

Wk: 18

→ RISC & CISC :

CISC	RISC
* Ins - more clock cycles	fewer clock cycles
* Ins Count - less Ins	More Ins
* Ins length - Variable	Fixed

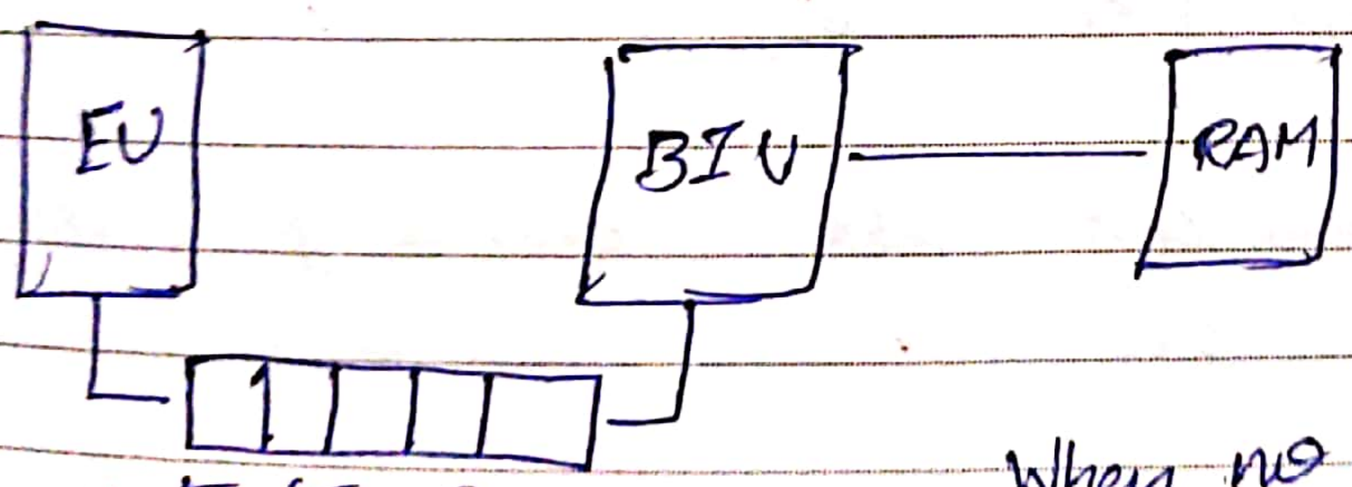
Inc Ax, - 1 byte
Mov Ax, DS[AX+100] - 8 byte

2

64 X 4

→ Execution

256 X 4



Instruction Queue of Buffer → When no instructions, BIU will fillit up.

[6 Bytes in x8086]

Fetches when 2 Bytes are free, carry 2 Bytes

5 word size

JUNE 2016

M	T	W	T	F	S	S	M	T	W	T	F	S	S
		1	2	3	4	5	6	7	8	9	10	11	12
13	14	15	16	17	18	19	20	21	22	23	24	25	26
27	28	29	30										

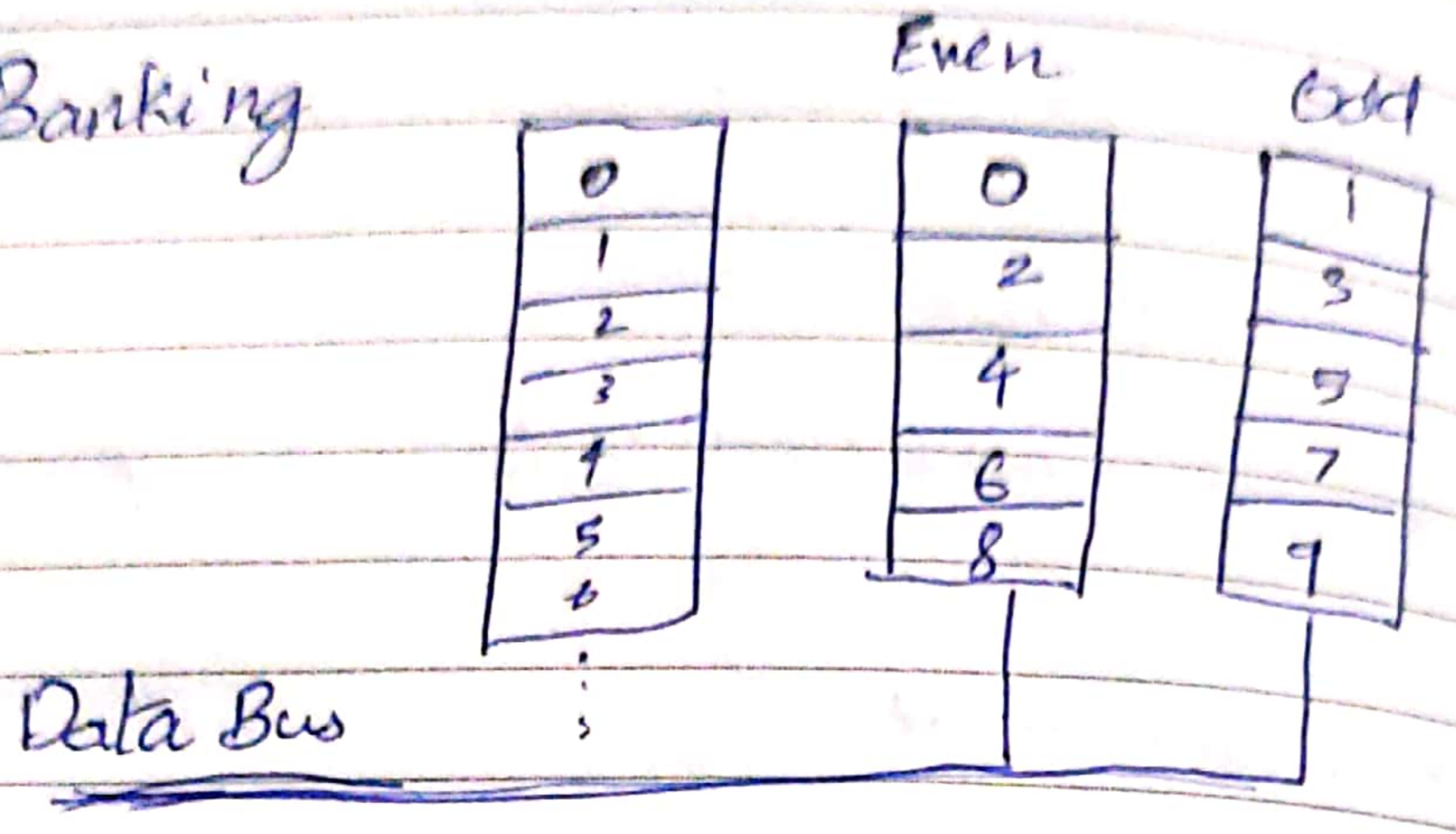
0205

3

May' 16 Tuesday

Wk 18

→ Memory Banking



With MOV AX, [even addr]

↳ This instruction uses even addr, even addr+1 for storage

With MOV AL, [odd addr] → Takes just odd address & save for one

With MOV AX, [odd addr]

It will do one [odd-1], odd - once to get odd
even [odd+1], odd+2 - once to get odd+1

It will discard the other values but it will take 2 × the time doing it with even address.

→ Pipelining:

```
F   D   E   M   WB
    F   D   E   M   WB
        F   D   E   M   WB
            F   D   E   M   WB
                F   D   E   M   WB
```

i_1: $R_1 \rightarrow R_2 + R_3$
i_2: $R_4 \rightarrow R_1 + R_5$
i_3: ...
i_4: ...

stall bubbles

```
F   D   E   M   WB
    F   D   _   _   E   M   WB
            F   D   _   E   M   WB
```

Since we have a write back, we need to wait until its done. In a large scale implementation we swap the order of instructions to maximize execution time.

We still have a OP code to make the Processor wait to handle the write backs. Swaps made based on lot of factors

* Types of dependencies:

No Dependency	True Dependency (*)	Output Dependency
$R_1 \rightarrow R_3 + R_6$	$R_1 \rightarrow R_2 + R_3$	$R_1 \rightarrow R_2 + R_3$
$R_2 \rightarrow R_4 + R_5$	$R_4 \rightarrow R_1 + R_5$	$R_1 \rightarrow R_4 + R_5$

5

May' 16

Thursday

Wk: 18

The Prophet's Ascension (Miaraj Night) (UAE); Father's Day (Germany)

Anti Dependency

$R_1 \rightarrow R_2 + R_3$

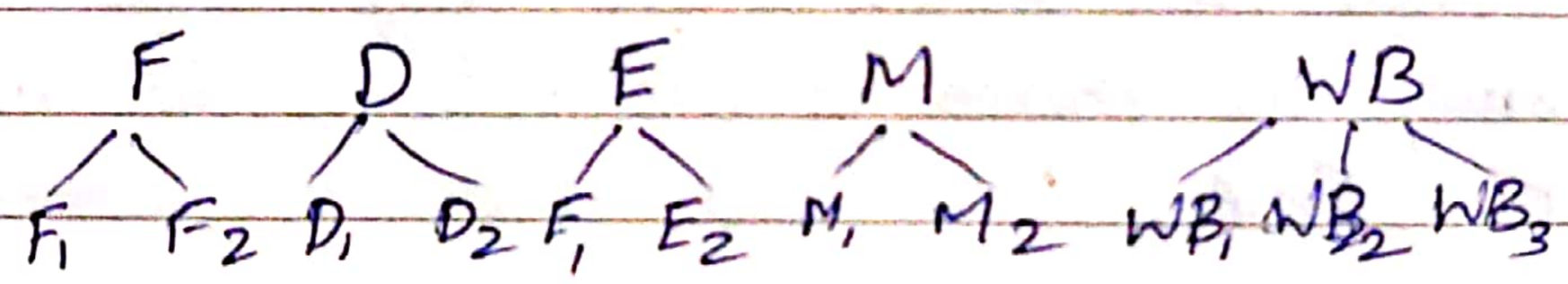
$R_2 \rightarrow R_4 + R_5$

Assignment - Read it

→ PIPELINE HAZARDS

* Control Hazard - when flow of ctrl changes
* Resource Conflict Hazard - when multiple instructions requires same resources.
* Structural Hazard

→ SUPER PIPELINE [VLIW] Very Large Instruction Word



VLI - Very Large Instruction

Here we overlap each of the sub stages to reduce time taken in large clock cycle instructions.

* Overlapping of stages

F_1 F_2 D_1 D_2 . . . .

F_1 F_2 D_1 . . .

Friday May' 16

We can even do this for Execution Stage

E_1 E_2 E_1 → this is based on the condition we dont use same condition.

Here we try to maximise overlapping to acheive maximum efficiency

* Super Scalar Pipeline

i_1	F	D	E	M	WB
i_2	F	D	E	M	WB
i_3		F	D	E	M	WB
i_4		F	D	E	M	WB

This had a lot issues, until they discovered it use cases of a multi core CPU.

→ MICRO INSTRUCTION :

In Order		Out-of-order
i_1		i_2
i_2		i_4
i_3	⇒	i_1
i_4		⋮
⋮		

JUNE 2016

M	T	W	T	F	S	S	M	T	W	T	F	S	S
		1	2	3	4	5	6	7	8	9	10	11	12
13	14	15	16	17	18	19	20	21	22	23	24	25	26
27	28	29	30										

06 05

Scanned with CamScanner

7 May' 16 Saturday

Wk: 18

In CISC, a single microinstruction does a elementary operation such as moving a value to a memory. So a single CISC Instruction Mass n microinstructions.

(Done by Assembly)

→ Memory Map: [Same as DOS Mem map for x8086]
We use a memory map (standard) that defines which parts/locations are used for what, its to be generic and let other applications run without causing problems.

Some eg: Like overwrite a app's code segment

8 Sunday

Wk: 18

Victory Day (France), Mother's Day (Australia)

May' 16

→ Variables, Procedures & Stack:

When a higher-level program creates vars, we need to store it somewhere. We only have 4 segs, so we write variable to stack with the SP seg to do it.

Push & POP instructions automatically do it.

Push AX
Push BX
Pop AX → AX has val of BX.

db a → Declare byte a [A Directive]
(X) NOT INSTRUCTION

07.05
08.05

Monday May' 16

Wk: 19

Declare byte - just get a byte of memory allocated to be used, and it 'a' will be replaced by address of byte (?)

We can also declare arrays.

* Functions / Procedures:
  (*) No params here

CALL some_procedure_lable (PUSHES the ~~next~~ IP into stack)
Jumps to label

RET (POPS IP from stack)

* Procedure:

```
PROC label
....
....
ret
```

(*) If you push to stack & dont pop, IP will chaged and causes problems

There is no concept of scope or parameters, everything is globally available.

If procedure is in other code segment, we need to Push/Pop both CS & IP. We use FAR Call, which does FAR Jump.

JUNE 2016

M	T	W	T	F	S	S	M	T	W	T	F	S	S
		1	2	3	4	5	6	7	8	9	10	11	12
13	14	15	16	17	18	19	20	21	22	23	24	25	26
27	28	29	30										

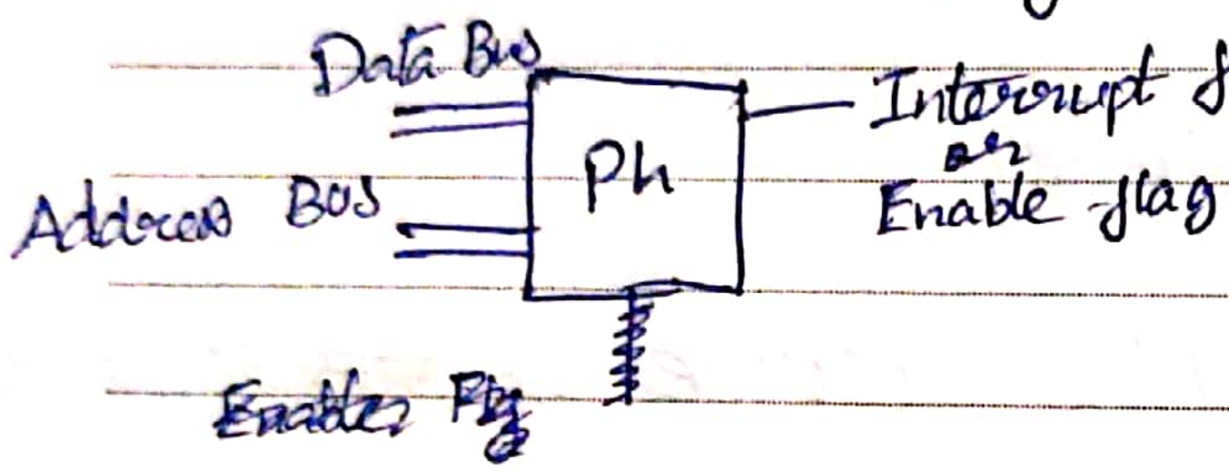
09.05

(X) x 8086 uses same lines for DB/AB
Data Bus / Address Bus.

→ Interrupts:

* To handle Peripherals.

We reuse DB/AB to connect these devices and we have a enable flag



CPU

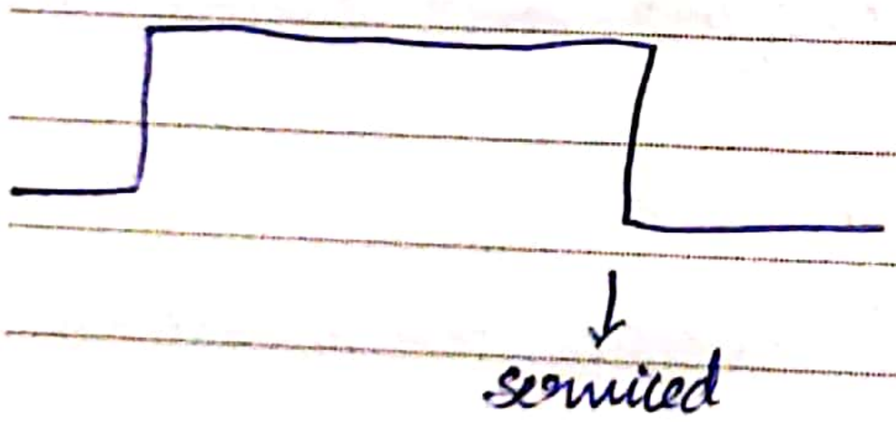
One bit line from Address Bus to send Interrupt/Signal ?

[Another for Non-Maskable Interrupt]

* TRIGGER INTERRUPT/EDGE INTERRUPT:

A sudden spike on the interrupt line. (Might be ignored by CPU).

* LEVEL TRIGGERED INTERRUPT:



A continuous hold on a the interrupt line [Separate one for level-triggered] until the device is serviced. (Generally handled by Priority for multi dev)

→ Masking Interrupts: [INTR / Maskable]

We use a flag/register to enable or disable certain Hardware interrupts. [Only 1 in 8085]

Disabled → Ignore

Enabled → Handle with Acknowledgement & ISR

→ Non-Masking Interrupt [NMI]:
A single pin, non-maskable, hardware interrupt that can't be disabled or delayed. Generates a TYPE 2 Signal in 8086.

All above Interrupts is Hardware Interrupts. Directly for Hardware.

→ SOFTWARE INTERRUPTS:
Instructions that generate interrupts. 256 Possible ones in 8086. They are of INT (Interrupt Type) First Byte OPcode, Second Byte is type number [2 Bytes]

IP loaded from type number * 04H & CS from (type * 04H) + 02H.

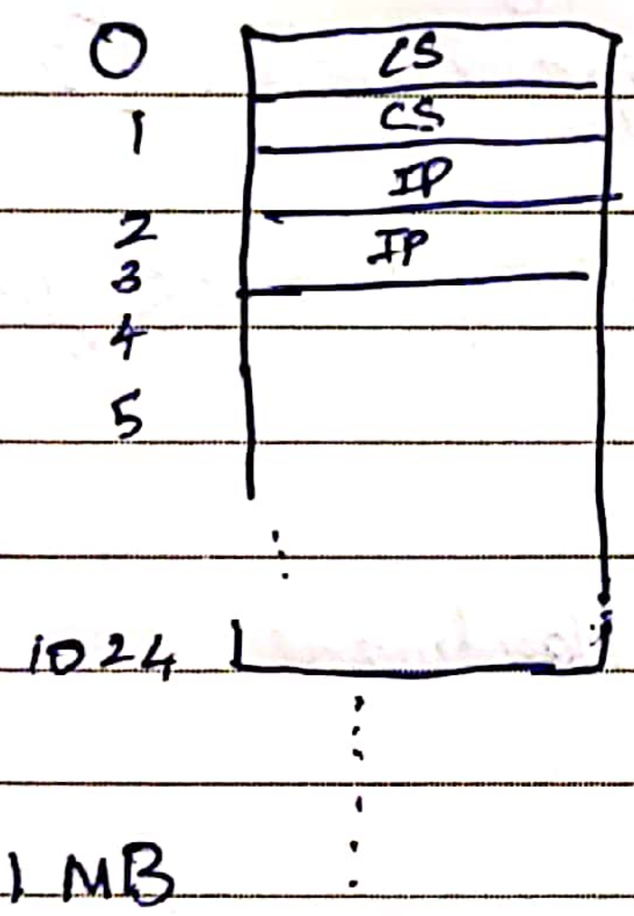
Internal Interrupt - Software & Hardware
External Interrupt - Hardware

→ INTERRUPT SERVICE ROUTINE [ISR]: 256 in 8086
They are blocks of code to which CPU jumps to when a interrupt is ~~den~~ ~~found~~ detected. It is done on the circuit level to be efficient. So when a IRQ makes PC change while preserving the state to not lose context of current execution

IRQ's make a special call, that pushes the state/ Flag Register of last instruction onto it. 'iret' is a special instruction, that will pop CS, IP & Flag Reg from Stack too, we use iret for ISR.

(happens implicitly)

Each IRQ has a numbers from 0-256. So In MEM 0-3 Bytes [CS+IP] for IRQ0, 4-7 - IRQ1 and so on. So 0,1 is CS & 2,3 is IP for IRQ0. We use a total of 1024 Bytes [256 X 4 Bytes]



This is IVT.

Interrupt Vector Table.

int → Interrupt

* Internal Interrupt:

(i) SW → calling Interrupts in programs to access hardware.

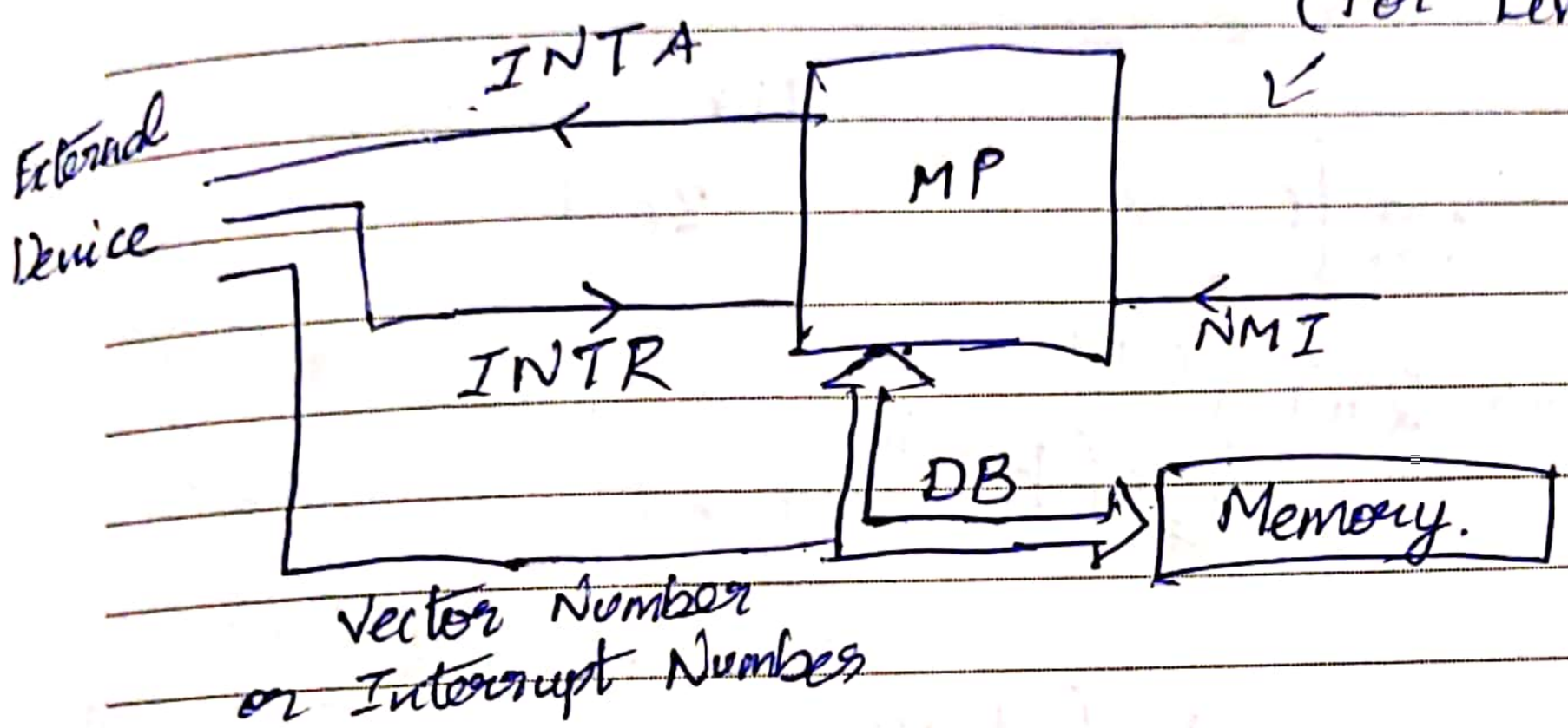
(ii) HW → Eg: While dividing if divisor is 0. We raise a interrupt.

* External " :

Generally from a External Device, for servicing

Each Interrupt has a Interrupt Number which mapped to a location in IVT leading to the ISR.

(For Level Triggered.)



Vector Number given in DB, we cant use memory at that point. ISRs are like drivers in Binary form. We need have a ISR for a interrupt, else the result is unpredictable (due to garbage code).

* Non - Vectored :

when INT -> Its directly mapped to a particular location in IVT by hard wiring. The ISR there polls each device according to priority. When a device is found to have a interrupt, its respective ISR is executed.

* Vectored :

when INT -> MP sends ACK to highest priority device, if no interrupt, ACK gets transmitted to next device and so on. When a device is found, it responds by placing ISR address in DB.